

HACKING THE PLANET

LINGUAGEM C



4

LISTA DE FIGURAS

Figura 1 – Índice de popularidade das Linguagens de Programação	7
Figura 2 – Importância da linguagem C	8
Figura 3 – Tela saída do Código-fonte Exemplo de uso da função printf para apresentar mensagem e variáveis em C	14
Figura 4 – Tela saída do Código-fonte Exemplo de uso da função scanf	17
Figura 5 – Representação de vetores	25
Figura 6 – Representação de string	26
Figura 7 – Tela de saída do Código-fonte Exemplo strings e funções de entrada e saída	27
Figura 8 – Saída do Código-fonte Exemplo strings e funções strcpy(), strcat(), strcmpi() e strlen()	29
Figura 9 – Representação de uma variável contendo um registro de um aluno	30
Figura 10 – Tipos diferentes de campos de registro para representar um caderno ..	31
Figura 11 – Representação de um vetor de registros de alunos	34
Figura 12 – Representação de uma variável na memória RAM	36
Figura 13 – Representação de uma variável ponteiro	37
Figura 14 – Representação de uma variável sendo alterada por seu ponteiro	38
Figura 15 – Tela de saída do Código-fonte Programa exemplo de declaração e uso de ponteiro para dado inteiro	38
Figura 16 – Exemplo de esquema de ponteiro apontando para uma variável do tipo struct	40
Figura 17 – Representação de função delta usada no programa exemplo	42
Figura 18 – Representação da função main antes da chamada da função delta	43
Figura 19 – Representação da função delta quando inicia sua execução delta	44
Figura 20 – Representação de parâmetros de entrada e saída de uma função	46
Figura 21 – Passagem de parâmetros de entrada (por valor) e saída (por referência)	47
Figura 22 – Representação de parâmetros de entrada e saída de uma função exemplo	47
Figura 23 – Esquema da passagem de parâmetros para função exemplo	48
Figura 24 – Esquema do retorno da passagem de parâmetros para função exemplo	49
Figura 25 – Esquema da passagem de parâmetros por valor e por referência para função delta	50
Figura 26 – Representação passagem de parâmetros das funções le_registro() e escreve()	51
Figura 27 – Representação declaração de vetor e de ponteiro pelo nome do vetor ..	53
Figura 28 – Representação da passagem de parâmetros do tipo vetor para função soma_vetor	53
Figura 29 – Representação da passagem de parâmetros para funções	54

LISTA DE QUADROS

Quadro 1 – Palavras-chave que não podem ser usadas como identificadores.....	11
Quadro 2 – Principais tipos de dados em C e seus tamanhos típicos.....	11
Quadro 3 – Formatos para a função printf apresentar os tipos de dados em C.....	13
Quadro 4 – Códigos especiais da função printf()	13

EXEMPLO

LISTA DE TABELAS

Tabela 1 – Operadores aritméticos da categoria binário	18
Tabela 2 – Operadores relacionais em C	19
Tabela 3 – Operadores lógicos em C	19
Tabela 4 – Representação de uma variável do tipo vetor de string	29

EXEMPLO

LISTA DE CÓDIGOS-FONTE

Código-fonte 1 – Estrutura básica de um programa em C	9
Código-fonte 2 – Exemplo de declarações de variáveis em C	12
Código-fonte 3 – Exemplo de uso da função printf para apresentar mensagem e variáveis em C.....	14
Código-fonte 4 – Exemplo de uso das operações aritméticas e suas prioridades	15
Código-fonte 5 – Exemplo simples de uso da função scanf() para ler variável inteira	15
Código-fonte 6 – Exemplo de uso da função scanf para ler vários tipos de variáveis	16
Código-fonte 7 – Exemplo de uso das funções getchar() e getch().....	18
Código-fonte 8 – Exemplo de uso da estrutura de controle com if	20
Código-fonte 9 – Exemplo de uso da estrutura de controle com switch case	21
Código-fonte 10 – Exemplo de uso do comando de repetição com do...while	22
Código-fonte 11 – Exemplo de uso do comando de repetição com while	23
Código-fonte 12 – Exemplo de uso do comando de repetição com for	24
Código-fonte 13 – Exemplo de vetores em C.....	25
Código-fonte 14– Exemplo strings e funções de entrada e saída	27
Código-fonte 15 – Exemplo strings e funções strcpy(), strcat(), strcmpi() e strlen().....	28
Código-fonte 16 – Exemplo de programa com vetor de strings (nomes)	29
Código-fonte 17 – Exemplo de programa para declaração de struct e seu uso (struct caderno)	32
Código-fonte 18 – Exemplo 2 de programa para declaração de struct e sua manipulação	33
Código-fonte 19 – Programa exemplo de vetor com registros (struct) de 3 alunos...	35
Código-fonte 20 – Programa exemplo de declaração e uso de ponteiro para dado inteiro	38
Código-fonte 21 – Programa exemplo de declaração de uso e ponteiro para dado float	39
Código-fonte 22 – Programa exemplo de declaração de uso de ponteiro para dado struct	39
Código-fonte 23 – Estrutura básica de uma função em C	41
Código-fonte 24 – Programa com função que calcula o discriminante de equação do 2º grau.....	43
Código-fonte 25 – Programa com função que verifica situação de peso	45
Código-fonte 26 – Função com passagem de parâmetros por valor e por referência.	47
Código-fonte 27 – Função delta com passagem de parâmetros por valor e por referência	49
Código-fonte 28 – Programa com funções com passagem de parâmetros do tipo struct	52
Código-fonte 29 – Programa com passagem de parâmetros do tipo vetor	54
Código-fonte 30 – Segundo programa com passagem de parâmetros do tipo vetor	56

SUMÁRIO

LINGUAGEM C	7
1 INTRODUÇÃO	7
2 CONSTRUÇÕES BÁSICAS E TIPOS DE DADOS	9
2.1 Programando com a linguagem C	9
2.2 Declaração de variáveis e tipos de dados	10
2.3 Apresentação de resultados do programa	12
2.4 Leitura de valores de dados via teclado	15
2.5 Construções básicas em C	18
3 TIPOS DE DADOS COMPOSTOS	24
3.1 Vetores e Matrizes	24
3.2 String: vetor de caracteres	26
3.3 Registros: estruturas de dados heterogêneas (struct)	30
3.4 Uso de vetor de struct	34
3.5 Variáveis do Tipo Ponteiro	35
4 FUNÇÕES	40
4.1 Funções com apenas 1 valor de resultado (return)	41
4.2 Funções com vários resultados (parâmetros de saída)	45
4.3 Funções com passagem de parâmetros do tipo de dado struct	50
4.4 Vetores, string e matrizes como parâmetros de funções	52
CONCLUSÃO	56
REFERÊNCIAS	57

LINGUAGEM C

1 INTRODUÇÃO

A compreensão de uma linguagem de programação é feita de modo gradual, dentro do que é conhecido como maturidade computacional, para que seja possível a resolução de problemas cada vez mais complexos. É preciso, antes de tudo, fazer uma escolha da linguagem mais apropriada, ou aquela que seja mais utilizada pela comunidade de desenvolvedores. Neste sentido, a busca por algum indicador é importante para determinar escolhas e saber se a linguagem utilizada para a aprendizagem tem ainda espaço e popularidade dentre a comunidade de desenvolvimento.

O indicador TIOBE, que pode ser observado na figura (Índice de popularidade das Linguagens de Programação), tem a motivação de medir a popularidade das linguagens de programação, com uma atualização mensal. As suas classificações são baseadas no número de profissionais qualificados em todo o planeta, cursos e fornecedores. Alguns mecanismos de busca populares como Google, Bing, Yahoo!, Wikipedia, Amazon, YouTube e Baidu são usados para calcular as classificações.

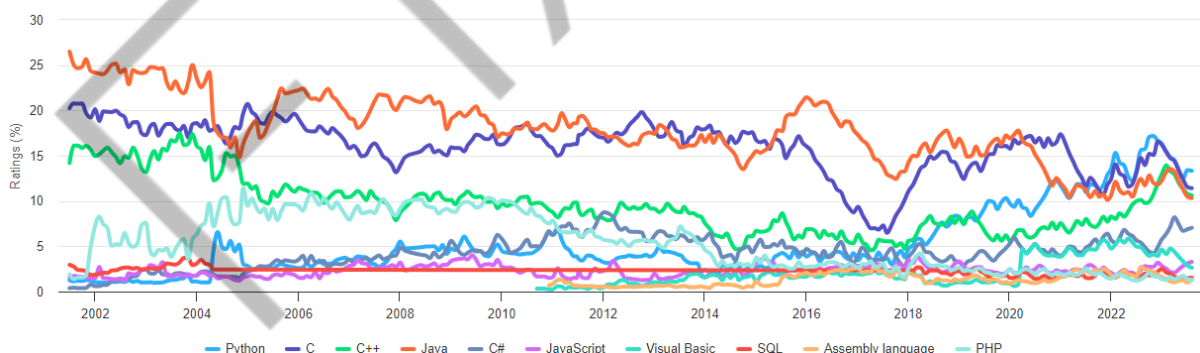


Figura 1 – Índice de popularidade das Linguagens de Programação
Fonte: Tiobe (2023)

Além da estabilidade no ranking por décadas, a linguagem C, dentro do universo de dispositivos conectados, é largamente utilizada na programação de microcontroladores como o Atmel AVR, que por sua vez compõe a plataforma de prototipagem eletrônica de hardware livre conhecida como Arduino. A linguagem C é muito utilizada para programar as estruturas de dados e opera de forma próxima ao hardware.

Apresentada a importância do aprendizado de uma linguagem de programação somado com a grandeza da linguagem C, o próximo passo é o de apresentá-la formalmente, por meio de sua sintaxe básica, sem a pretensão de que você seja um exímio programador ou uma brilhante programadora.

A linguagem C tem grande vantagem com relação a outras linguagens. A Figura 2” apresenta a resposta do professor Waldemar Celes, da PUC-Rio e coordenador de projetos na área de Visualização do Instituto Tecgraf/PUC-Rio em uma entrevista à CIO/IDG.



Figura 2 – Importância da linguagem C
Fonte: Celes (2016)

Ainda segundo Celes (2016):

- Linguagens de programação mais abstratas oferecem suporte nativo às estruturas de dados mais usuais; em C, precisamos implementá-las.
- O conhecimento adquirido ao implementar as estruturas nos faz ter um melhor discernimento de qual estrutura devemos usar numa determinada aplicação, além de nos capacitar a adaptar estruturas já existentes quando necessário, e também nos ensina a avaliar a eficiência dos algoritmos que fazem uso dessas estruturas.

Dadas as devidas considerações, o próximo passo é apresentar a estruturação básica de sintaxe da linguagem.

2 CONSTRUÇÕES BÁSICAS E TIPOS DE DADOS

2.1 Programando com a linguagem C

Inicialmente você irá recordar conceitos de algoritmos e linguagens de programação já conhecendo como são implementados usando a linguagem. Em seguida, serão apresentadas novas informações, definições e comandos da linguagem C, necessários para programar de forma eficiente as estruturas de dados abordados ao longo do texto.

Em termos de estrutura mínima, um programa escrito em linguagem C tem a seguinte configuração:

```
#include <stdio.h>
// várias bibliotecas podem ser incluídas
int main() // função principal
{
    /*
     declaração de variáveis
    */

    //comandos da linguagem C
}
```

Código-fonte 1 – Estrutura básica de um programa em C
Fonte: Elaborado pelo autor (2020)

O identificador `#include` é uma das chamadas diretivas de pré-processamento. Uma diretiva não é um comando da linguagem C e sim um “comando” do pré-processador, o qual, antes de compilar o programa, examina o código realizando modificações conforme as diretivas.

A ação gerada pelo pré-processador pela diretiva `#include` é inclusão (ou importação) de bibliotecas com funções específicas. Por exemplo, quando forem utilizadas funções de entrada de dados via teclado e saída de resultados na tela de saída do programa, será preciso incluir o arquivo de biblioteca denominado `stdio.h` (*standart input/output*), a fim de incluir as definições das funções de entrada e saída `printf()` e `scanf()`, que estão contidas nessa biblioteca do compilador C. O pré-processador faz com que o arquivo `stdio.h` seja inserido no atual programa.

Todos os arquivos que pertencem à biblioteca da linguagem C têm como extensão “.h”, e conforme os programas apresentados tenham funções de outras bibliotecas, o conhecimento sobre elas aumenta de forma exponencial.

Observe que após o `#include` aparece o nome da biblioteca entre os delimitadores '`<`' e '`>`'. E caso outras bibliotecas sejam necessárias, deverá ser incluída seguindo a mesma regra e um `#include` `<` `>` em cada linha.

O código a ser executado é colocado, como indicado, entre os símbolos `{` e `}`, que demarcam o início e o final (respectivamente) da função principal sempre denominada `main()`.

Quando um programa é escrito, sempre se tem a recomendação de colocar comentários para identificar soluções e observações que possam ajudar a entender e modificar o código após um longo período sem utilizá-lo. E falando-se em comentários, os mesmos podem ser introduzidos no código de duas formas:

- Usando `//` que permite inserir um comentário até o final da linha.
- Através dos delimitadores `/*` e `*/` para início e final de comentário, respectivamente, assim várias linhas podem ser consideradas como comentários.

Logo no início da função `main` devem ser declaradas todas as variáveis e, finalmente, é feita a inserção dos comandos a serem executados no programa.

2.2 Declaração de variáveis e tipos de dados

Nos programas, os dados são armazenados em variáveis. Em C, toda variável deve ser associada a um tipo de dado que irá armazená-lo, o que é feito na *declaração* da variável.

Como fazer a declaração de uma variável? Por intermédio de um tipo ou identificador, nomes formados por uma sequência de letras ou dígitos, sendo que o primeiro caractere deve ser uma letra: o caractere “`_`” é considerado uma letra, e letras maiúsculas e minúsculas são consideradas distintas (*case sensitive* ou sensível a maiúscula ou minúscula).

E existe um conjunto de identificadores reservados para a linguagem C, tais nomes não poderão ser utilizados como nomes de variáveis de usuário. Tais identificadores são denominados palavras reservadas, que seguem como uma lista no quadro a seguir.

Palavras chaves em C (padrão ANSI)			
auto	double	int	struct
break	else	long	switch
case	enum	register	typedef
char	extern	return	union
const	float	short	unsigned
continue	for	signed	void
default	goto	sizeof	volatile
do	if	static	while

Quadro 1 – Palavras-chave que não podem ser usadas como identificadores
Fonte: Elaborado pelo autor (2020)

Quanto aos tipos de dados existentes na linguagem C, os principais que serão utilizados neste momento inicial são apresentados a seguir, no quadro “Principais tipos de dados em C e seus tamanhos típicos”:

Tipo	Descrição	Tamanho típico (bytes)
char	Caractere	1
int	Inteiro com sinal	4 ou 8
unsigned int	Inteiro sem sinal	4 ou 8
long int	Inteiro longo	8 ou 16
float	Real	8
double	Real longo	16
void	Vazio	0

Quadro 2 – Principais tipos de dados em C e seus tamanhos típicos
Fonte: Elaborado pelo autor (2020)

O programa a seguir, representado pelo Código-fonte “Exemplo de declarações de variáveis em C”, exemplifica a declaração de variáveis em um programa. Observe

que é possível apenas declarar uma variável, e posteriormente atribuir um valor a ela, ou, então, atribuir um valor inicial quando ocorre a sua respectiva declaração.

```
#include <stdio.h>

int main()
{
    int n;
    float x, pi=3.14;
    unsigned int contador = 0;
    char letra;
    n = -1;
    x = 2.3;
    letra = 'A' ;
}
```

Código-fonte 2 – Exemplo de declarações de variáveis em C
Fonte: Elaborado pelo autor (2020)

2.3 Apresentação de resultados do programa

Para poder escrever, ou seja, apresentar resultados na tela de saída será utilizada a função `printf()`, uma das possibilidades dentro da linguagem. Esta função encontra-se definida na biblioteca `<stdio.h>`. É uma função que não apenas apresenta mensagem e valores de variáveis na tela de saída, mas também é capaz de formatar os valores dos dados antes de apresentá-los (número de casas decimais por exemplo). Veja um exemplo de sua sintaxe:

```
printf("expressão de controle", lista de argumentos);
```

A expressão de controle pode ser uma mensagem constante ou um código que determina o formato do argumento (variável) que deve ter seu conteúdo escrito na tela de saída. Os códigos de formatação do comando `printf` podem ser vistos no quadro a seguir:

Código	Formato
%c	Caracter simples
%d	Decimal
%e	Notação científica
%f	Ponto-flutuante (real)
%g	%e ou %f (o mais curto)
%o	Octal
%s	Cadeia de caracteres
%u	Decimal sem sinal
%x	Hexadecimal
%ld	Decimal longo
%lf	Double

Quadro 3 – Formatos para a função printf apresentar os tipos de dados em C
 Fonte: Elaborado pelo autor (2020)

Além do formato, alguns códigos especiais podem ser utilizados:

Códigos Especiais	Significado
\n	Nova linha
\t	TAB
\""	Aspas
\\	Barra
\o	Nulo

Quadro 4 – Códigos especiais da função printf()
 Fonte: Elaborado pelo autor (2020)

No Código-fonte 3 – “Exemplo de uso da função printf para apresentar mensagem e variáveis em C” é possível observar como a linguagem utiliza a função printf para a formatação da saída na tela, inicialmente fazendo a declaração das variáveis e armazenamento de valores de acordo com os seus respectivos tipos.

```
#include <stdio.h>

int main()
{
    int x;
    float y;
```

```

char z;
x=1;
y= 2.3;
z='A' ;
printf("o valor de x = %d\n", x);
//formato %d para valor inteiro
printf("valor de y = %f\n", y);
//formato %f para valor float
printf("%c\n", z);
//formato %c para valor caractere

y=8.5421;
printf("x somado a 1 = %d \n e z = %c\n", x+1,z);
printf("\n%0.2f\n", y); /* %0.2f formata saída para
quantidade de dígitos livre para parte inteira e 2 dígitos
para parte fracionaria */
}

```

Código-fonte 3 – Exemplo de uso da função printf para apresentar mensagem e variáveis em C
 Fonte: Elaborado pelo autor (2020)

A Figura 3a seguir apresenta a tela de saída correspondente, indicando o momento em que as formatações numéricas foram aplicadas:

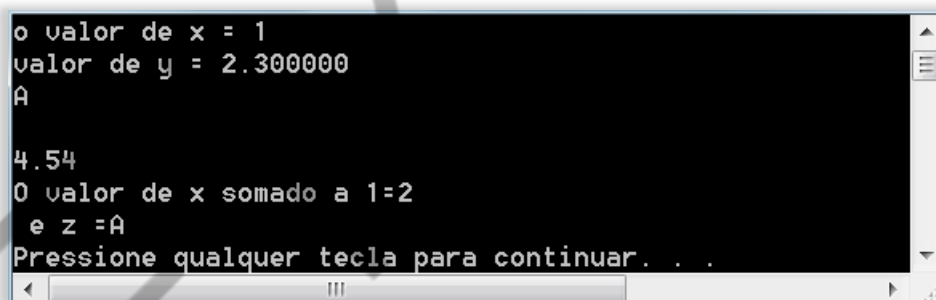


Figura 3 – Tela saída do Código-fonte Exemplo de uso da função printf para apresentar mensagem e variáveis em C
 Fonte: Elaborado pelo autor (2020)

O programa a seguir será utilizado para observar de perto as funções de entrada e saída assim como para apresentar alguns operadores aritméticos e suas regras de precedência.

```

#include <stdio.h>
int main()
{
    float x,y,z,w; //variáveis x,y,z e w são do tipo real
    int m, n=1;    //variáveis m e n são do tipo inteiro
    x=3.2;         // variável x recebe o valor 3.2
    y=x*2;         //variável y recebe o valor 6.4
    z=(x+y)/2;     // variável z recebe o valor 4.8
    w= x+y/2;      // variável w recebe o valor 6.4
}

```

```
m = n/2;    // variável m recebe o valor 0
x = n/2;    // variável x recebe o valor 0.0
x = n/2.0;  // variável x recebe o valor 0.5
printf("x=%0.1f \t y=%0.1f \n", x, y);
printf("z=%0.1f \t w=%0.1f \n", z, w);
printf("m=%d \t n=%d \n", m, n);
}
```

Código-fonte 4 – Exemplo de uso das operações aritméticas e suas prioridades
Fonte: Elaborado pelo autor (2020)

DICA: Para que você possa praticar um pouco, utilize o seguinte endereço para fazer os testes e visualizar os resultados apresentados, por meio de um compilador online: <https://www.onlinegdb.com/online_c_compiler>

2.4 Leitura de valores de dados via teclado

A função nativa `scanf()` foi implementada para permitir a entrada de dados em um programa escrito na linguagem C. Este comando encontra-se definido na biblioteca “`stdio.h`”, e sua sintaxe é:

```
scanf("expressão de controle", lista de argumentos);
```

Assim como no `printf`, a função `scanf()` não apenas lê um valor digitado no teclado e armazena em uma variável do programa, mas também realiza a conversão de tipo em função da especificação do programador(a). Tal especificação deve ser realizada na expressão de controle, por meio de um código que determina o formato do argumento (variável) que se deseja ter seu conteúdo lido do teclado. Os códigos de formatação são os mesmos da função `printf()`.

Observe a seguir um exemplo muito simples que pede para o usuário inserir um valor inteiro, e neste caso é preciso utilizar a tecla **[ENTER]** para indicar que o valor foi inserido.

```
#include <stdio.h>
int main()
{
    int x;
    printf("digite um valor inteiro: ");
    scanf("%d", &x);
    printf("o valor de x = %d\n", x);
}
```

Código-fonte 5 – Exemplo simples de uso da função `scanf()` para ler variável inteira
Fonte: Elaborado pelo autor (2020)

É importante notar que a função `scanf` exige uma referência à variável que armazena o valor lido do teclado, usando-se o operador `&`. O entendimento mais completo desse operador será necessário quando forem utilizadas variáveis do tipo ponteiro. Nesse momento, é suficiente saber que, por exemplo, `&x` especifica que o valor lido do teclado como número inteiro (formato `%d`) é armazenado no endereço de memória RAM (*Random Access Memory* ou Memória de Acesso Aleatório) reservado para guardar a variável `x`.

IMPORTANTE: na expressão de controle do `scanf` deve apenas ter o(s) formato(s) da(s) variável (variáveis) e nenhum caractere a mais, nem espaço em branco, pois irá ocasionar um erro de sintaxe.

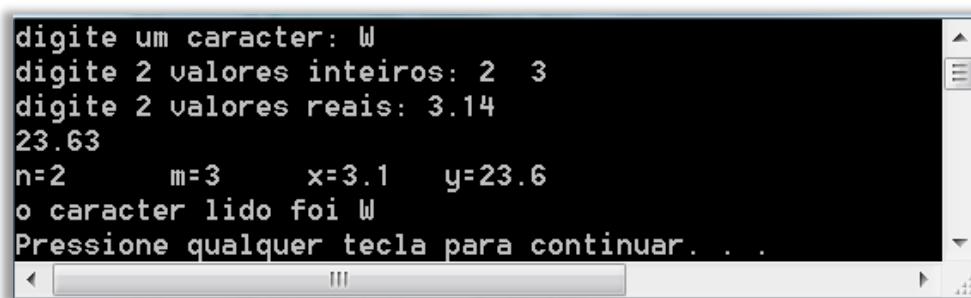
O código-fonte a seguir mostra como utilizar `scanf()` em variáveis de vários tipos, mostrando as possibilidades de utilização.

```
#include <stdio.h>
int main()
{
    int n, m;
    float x,y;
    char letra;
    printf("digite um caracter: ");
    scanf("%c", &letra);
    printf("digite 2 valores inteiros: ");
    scanf("%d%d", &n, &m);
    printf("digite 2 valores reais: ");
    scanf("%f", &x);
    scanf("%f", &y);
    printf("n=%d \t m=%d \t x=%0.1f \t y=%0.1f \n", n, m, x, y);
    printf("O caracter lido foi %c", letra);
}
```

Código-fonte 6 – Exemplo de uso da função `scanf` para ler vários tipos de variáveis

Fonte: Elaborado pelo autor (2020)

Observe a saída de tela apresentada na Figura “Tela saída do Código-fonte Exemplo de uso da função `scanf`”, simbolizando a saída de tela para um caso particular.



```
digite um caracter: W
digite 2 valores inteiros: 2 3
digite 2 valores reais: 3.14
23.63
n=2      m=3      x=3.1    y=23.6
o caracter lido foi W
Pressione qualquer tecla para continuar...
```

Figura 4 – Tela saída do Código-fonte Exemplo de uso da função scanf
Fonte: Elaborado pelo autor (2020)

É importante notar que no exemplo `scanf("%d%d",&n,&m)` não há espaço em branco e nem qualquer tipo de caractere.

DICA: se de forma inadvertida for escrito qualquer caractere na expressão de controle, irá notar que a função `scanf()` não irá ler o dado do teclado passando como se não tivesse sido executada.

Para realizar a leitura do teclado e obter caracteres, pode ser usada a função `getchar()`, sendo que o caractere será lido somente quando pressionada a tecla [ENTER]. A sua sintaxe é:

```
letra = getchar();
```

No código-fonte a seguir, note que foi usada a função `fflush(stdin)`, para esvaziar o buffer (memória temporária) direcionada para guardar cada tecla pressionada do teclado (`stdin`- *standard input* – entrada padrão). Apenas na leitura de caracteres é necessário executar tal função, como também mostra o código. Caso seja esquecido de inserir a chamada da função `fflush(stdin)`, o que irá ocorrer é que o programa não fará a leitura do teclado do caractere, sendo assim, logo perceberá que deve corrigir o código. Como parte do aprendizado, procure remover as chamadas da função `fflush(stdin)` do programa e observe o que ocorre.

```
#include <stdio.h>
int main()
{
    int x;
    char letra;
    printf("digite uma letra: ");
    fflush(stdin);      //limpa buffer do teclado
    letra= getchar();
    printf("digite um numero inteiro: ");
    scanf("%d",&x);
```

```
fflush(stdin);          //limpa buffer do teclado
printf("%c", letra);
printf("%d", x);
}
```

Código-fonte 7 – Exemplo de uso das funções getchar() e getche()
Fonte: Elaborado pelo autor (2020)

2.5 Construções básicas em C

Toda e qualquer linguagem de programação possui como parte integrante de sua sintaxe básica os operadores, e os primeiros a serem apresentados são os aritméticos (+, -, *, / e %), os relacionais (como por exemplo >, <=, ==, !=) e lógicos (como por exemplo ||, && e !).

Os operadores aritméticos são usados para calcular expressões matemáticas, e classificados em duas categorias: binários e unários. Os operadores unários atuam na inversão de valores, e utilizam os símbolos – e +. A Tabela abaixo mostra os operadores pertencentes à categoria binário, por justamente depender de dois elementos para a operação:

Operação	Operador
Soma	+
Subtração	-
Multiplicação	*
Divisão	/
Módulo (resto)	%

Tabela 1 – Operadores aritméticos da categoria binário
Fonte: Elaborado pelo autor (2020)

Nos operadores relacionais, como o próprio nome induz, existe uma relação de comparação entre os elementos, indicando por exemplo maior ou menor que, diferente ou igual, sendo largamente utilizados ao longo dos códigos para a resolução de problemas.

É possível observar os operadores relacionais através da Tabela “Operadores relacionais em C” a seguir:

Símbolo	Operador	Exemplo	Significado
>	Maior que	$x > y$	x é maior que y ?
>=	Maior ou igual	$x \geq y$	x é maior ou igual a y?
<	Menor que	$x < y$	x é menor que y?
<=	Menor ou igual	$x \leq y$	x é menor ou igual a y?
==	Igualdade	$x == y$	x é igual a y?
!=	Diferente de	$x != y$	x é diferente de y?

Tabela 2 – Operadores relacionais em C
Fonte: Elaborado pelo autor (2020)

IMPORTANTE: é fundamental distinguir (=) de (==). O primeiro é de atribuição (aritmético) e o segundo compara expressões (relacional).

Os operadores lógicos servem para interligar mais de uma operação relacional, e do mesmo modo que os relacionais, retornam zero para falso e um para verdadeiro. A Tabela abaixo exemplifica os operadores lógicos utilizados pela linguagem C:

Operação	Operador
E	&&
OU	
NÃO	!

Tabela 3 – Operadores lógicos em C
Fonte: Elaborado pelo autor (2020)

Definidos os principais operadores da linguagem, é importante analisar a utilização das estruturas de controle, a qual permite que o programa faça a escolha do que executar, de acordo com uma condição. A linguagem C oferece comandos para a utilização com dois tipos de estruturas: *if...else* e *switch...case*.

Construção **if...else**:

```
if (condição)
{
    /* conjunto de comandos do se; */
}
else
{
    /*conjunto de comandos do senão; */
}
```

IMPORTANTE: não existem valores do tipo *boolean* (lógico) *TRUE* ou *FALSE*, pois, tais valores lógicos passam a ter um valor do tipo *int* correspondente à condição, sendo *TRUE*=1 e *FALSE*=0.

Para exemplificar o uso deste tipo de estrutura de controle, observe o código-fonte a seguir, em que uma operação de soma entre dois números inteiros é efetuada e em seguida é verificado se o valor da soma é maior que o número 10, uma decisão bem simples de ser executada.

```
#include <stdio.h>
#include <stdlib.h>
int main(void)
{
    int N1, N2, Soma;
    printf("Digite um numero inteiro: ");
    scanf("%d", &N1);

    printf("Digite um numero inteiro: ");
    scanf("%d", &N2);

    Soma = N1 + N2;
    printf("O Valor da soma = %d\n", Soma);

    if(Soma > 10)
    {
        printf("O valor da soma e maior que 10\n");
    }
    else
    {
        printf("Valor menor ou igual a 10\n");
    }
}
```

Código-fonte 8 – Exemplo de uso da estrutura de controle com if
Fonte: Elaborado pelo autor (2020)

Construção **switch ... case**:

```
switch (var) {
case opção1: comandos;
    break;
case opção2: comandos;
    break;
...
default: comandos;
}
```

A utilização da estrutura de controle com o switch...case passa a ser uma opção interessante para testar um determinado resultado de uma expressão ou valor atual de uma variável.

No Código-fonte “Exemplo de uso da estrutura de controle com switch case” é possível ver a aplicação da estrutura de controle switch...case, testando o valor da variável resp, e que no caso teve uma atribuição manual do valor 3, e quando é verdadeira a verificação, não será preciso testar as demais opções, pois o comando break executa a saída do bloco.

```
#include<stdio.h>
int main() {
    int resp;

    resp = 3;
    switch (resp)
    {
        case 1:
            printf("Voce selecionou a opcao %d", resp);
            break;

        case 2:
            printf("Voce selecionou a opcao %d", resp);
            break;
        case 3:
            printf("Voce selecionou a opcao %d", resp);
            break;

        default:
            printf("Voce selecionou uma opcao diferente de 1, 2
ou 3 --> %d", resp);
    }
}
```

Código-fonte 9 – Exemplo de uso da estrutura de controle com switch case
Fonte: Elaborado pelo autor (2020)

E quando houver a necessidade de repetição dentro do programa, existem estruturas especiais para que seja possível implementar na linguagem C, as chamadas estruturas de repetição. Tais estruturas permitem que um bloco de instruções seja executado, repetidamente, uma quantidade controlada de vezes. A linguagem C possui 3 estruturas de repetição: *do...while*, *while* e *for*.

A construção **do while** é uma estrutura básica de repetição condicional e permite a execução de um bloco de instruções repetidamente, sendo que o critério de parada é testado após a entrada no laço:

```
do
{
    /*comandos executados enquanto condição verdadeira*/
} while(condição)
```

O Código-fonte “Exemplo de uso do comando de repetição com do...while” mostra como o comando de repetição do...while funciona, apresentando como resultado a somatória de números somente após o usuário digitar o número 0.

```
#include <stdio.h>
int main()
{
    double number, soma = 0;

    do
    {
        printf("Insira um número: ");
        scanf("%lf", &number);
        soma += number;
    }
    while(number != 0.0);

    printf("Soma = %.2lf", soma);

    return 0;
}
```

Código-fonte 10 – Exemplo de uso do comando de repetição com do...while
Fonte: Elaborado pelo autor (2020)

A construção **while** faz com que a condição seja avaliada em primeiro lugar. Se a condição for verdadeira, o bloco é executado uma vez e a condição é avaliada novamente. Caso a condição seja falsa, a repetição é terminada sem a execução do bloco.

Observe que nesta estrutura, ao contrário da estrutura do...while, o bloco de instruções pode não ser executado nenhuma vez, basta que a condição seja inicialmente falsa.

```
while (condição)
{
    /*comandos executados enquanto condição verdadeira*/
}
```

```
}
```

A seguir, o código-fonte mostra a aplicação simples do comando `while`, em uma situação em que ocorre o incremento de uma unidade do número 10, até que ele tenha o valor 19, última iteração do laço, pois o critério de parada é estipulado para valores inteiros menores do que 20.

```
#include <stdio.h>

int main () {

    int num = 10;

    while( num < 20 ) {
        printf("Valor atual número: %d\n", num);
        num++;
    }

    return 0;
}
```

Código-fonte 11 – Exemplo de uso do comando de repetição com `while`
Fonte: Elaborado pelo autor (2020)

A construção **for** é muito semelhante às estruturas de repetição vistas anteriormente, entretanto costuma ser utilizada quando se quer um número determinado de ciclos. A contagem dos ciclos é feita por uma variável chamada de contador. A estrutura `for` é, às vezes, chamada de estrutura de repetição com contador. Sua sintaxe é a seguinte:

```
for(comando de inicializacao; teste lógico; comando 3)
{
    /*comandos a serem repetidos*/
}
```

Uma forma simples de contagem de números pode ser feita com o comando `for`, o qual pode ser observado pelo Código-fonte “Exemplo de uso do comando de repetição com `for`”, que efetua a contagem dos números de 1 até 10.

```
#include <stdio.h>
#include <conio.h>
int main(void)
{
    int contador; //variável de controle do loop
```

```
for(contador = 1; contador <= 10; contador++)  
{  
    printf("%d ", contador);  
}  
  
getch();  
}
```

Código-fonte 12 – Exemplo de uso do comando de repetição com for
Fonte: Elaborado pelo autor (2020)

3 TIPOS DE DADOS COMPOSTOS

Quando o armazenamento ou atribuição de valores às variáveis se torna mais complexo, faz-se a recursão à utilização de tipos de dados que agregam mais valores. E todo tipo de dado que agrega mais de um valor é dito que se trata de uma estrutura de dados. Na linguagem C são utilizados dois tipos de dados compostos: os que agregam elementos do mesmo tipo (vetor e matrizes) e os que são compostos por dados de tipos diferentes (*structs*).

3.1 Vetores e Matrizes

Como vetores (e matrizes) podem conter apenas valores do mesmo tipo, são conhecidos também como estruturas de dados homogêneas.

Para declarar uma variável do tipo vetor em C:

```
<Tipo de dado do elemento> nome_identificador [número de elementos];
```

Exemplos de declaração:

```
int x[5];  
float valores[7];
```

A Figura 5 “Representação de vetores” apresenta como são representados os vetores declarados. Cada elemento é identificado de forma única pelo índice. Em C, sempre os elementos de um vetor são numerados por índices de 0 até número de elementos – 1.



Figura 5 – Representação de vetores
Fonte: Elaborado pelo autor (2020)

Então para que se tenha acesso a um determinado elemento, é necessário especificar a variável do tipo vetor e entre colchetes determinar qual o índice do elemento, por exemplo, `x[2]`, `valores[0]`, etc.

Para realizar leitura e escrita de um vetor inteiro deve-se escolher elemento a elemento, por isso, normalmente é usada uma repetição como no Código-fonte “Exemplo de vetores em C” (Exemplo de vetores em C).

IMPORTANTE: constantes são usadas para armazenar valores que **NÃO** podem ser modificados durante a execução de um programa. Uma constante precisa ser declarada e, para tanto, usa-se a diretiva de pré-processador `#define`.

```
#include <stdio.h>
#define N 5
int main()
{
    int idade[N], i, cont=0;
    for (i=0; i<N; i++ ) {
        printf("digite a idade: ");
        scanf("%d", &idade[i]);
        if (idade[i] >= 18)
            cont++;
    }
    printf("numero de pessoas com idade acima 18 anos = %d\n",
        cont);
}
```

Código-fonte 13 – Exemplo de vetores em C
Fonte: Elaborado pelo autor (2020)

No programa é apresentado mais um novo componente de um programa em C, a diretiva `#define`. Como já foi mencionado anteriormente, uma diretiva é executada pelo pré-processador, neste caso a ação gerada será a definição da constante `N` (lembre-se que C diferencia letras maiúsculas de minúsculas) que terá o valor 5 atribuído a ela. Assim, quando o programa for compilado onde a constante `N` for

encontrada será trocada pelo valor 5. A vantagem de usar a definição de constante é que se for necessário trocar o tamanho do vetor de 5 para 10 basta alterar o valor da constante e todo o programa onde N é usado, o novo valor será usado.

Voltando ao código e utilização de vetor, observe que a estrutura de repetição `for` é usada para gerar índices que vão de 0 até 4, com isso em cada repetição a variável `i` usada pelo `for` terá o valor do índice.

Cuidado: programas em C não avisam quando o limite do dimensionamento de um vetor `for` excedido e podem gerar resultados errados.

3.2 String: vetor de caracteres

Não existe o tipo básico de dado “*string*” como existe em algumas linguagens. Uma string na linguagem C é um vetor de caracteres (elementos do tipo `char`).

Esse vetor de caracteres tem tratamento diferenciado:

1. O final da string armazenada no vetor é marcado com o caractere nulo ‘\0’ (barra zero);
2. Existem funções especiais para tratar esse tipo de dado que serão estudadas a seguir.

A variável que será uma string em C pode ser declarada como:

```
char nome[10];
```

Nesta declaração, `nome` é a variável do tipo vetor de caracteres (string) que poderá conter 10 caracteres, embora um deles sempre conterà o caractere ‘\0’ (Figura 6 “Representação de string”).



Figura 6 – Representação de string
Fonte: Elaborado pelo autor (2020)

Para realizar a leitura e escrita de uma string podem ser usadas as abordagens:

1. Usando funções básicas da biblioteca `stdio.h`:

```
scanf("%s",nome);
```

```
printf("%s", nome);
```

2. Usando funções especiais da biblioteca string.h:

```
gets(nome);
```

```
puts(nome);
```

O exemplo mostra como usar essas funções especiais:

```
#include <stdio.h>
#include <string.h>

int main()
{
    char nome1[30], nome2[30];
    printf("digite o nome: ");
    fflush(stdin);
    scanf("%s", nome1); //única exceção do não uso & no scanf

    printf("digite novamente o nome: ");
    fflush(stdin);
    gets(nome2); /*equivale ao scanf mas pode fazer
    a leitura do nome mesmo com espaço em branco */

    printf("\n O nome : %s\n", nome1);
    printf("\n O nome completo: %s\n", nome2);
}
```

Código-fonte 14– Exemplo strings e funções de entrada e saída
Fonte: Elaborado pelo autor (2020)

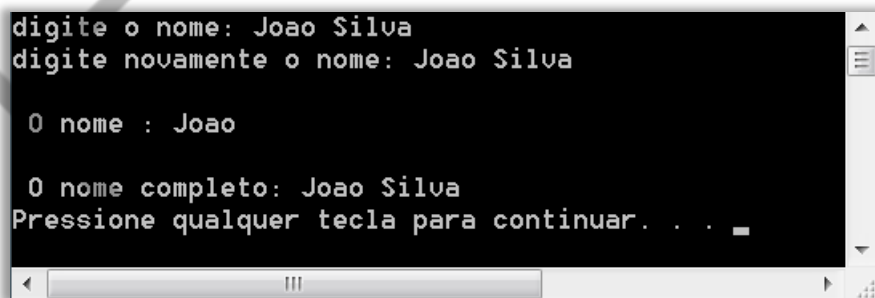


Figura 7 – Tela de saída do Código-fonte Exemplo strings e funções de entrada e saída
Fonte: Elaborado pelo autor (2020)

A função `scanf()` quando lê a informação digitada do teclado “Joao Silva” ignora o sobrenome, pois essa função considera o espaço em branco uma forma de finalizar uma string. Já a função `gets()` não segue essa mesma lógica e assim consegue ler o nome completo digitado.

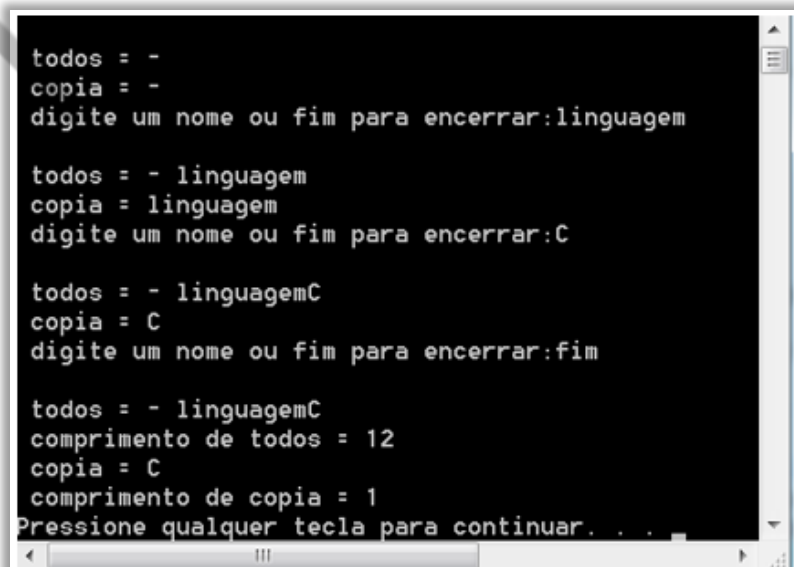
IMPORTANTE: a função `scanf()` considera que o espaço em branco finaliza uma string.

Existem outras funções especiais para manipular strings na biblioteca `<string.h>`. O programa a seguir mostra o uso de funções básicas para manipulação de *string*.

```
#include <stdio.h>
#include <string.h>

int main()
{
    char nome[30], todos[200], copia[30];
    strcpy(nome, "- ");
    while (strcmp(nome, "fim") != 0) {
        strcat(todos, nome);
        //concatena a variável nome a variável todos
        strcpy(copia, nome);
        //copia o conteúdo da variável nome 'a copia
        printf("\n todos = %s ", todos);
        printf("\n copia = %s ", copia);
        printf("\n digite um nome ou fim para encerrar:");
        fflush(stdin);
        gets(nome);
    }
    printf("\n todos = %s", todos);
    printf("\n comprimento de todos = %d", strlen(todos));
    printf("\n copia = %s", copia);
    printf("\n comprimento de copia = %d\n", strlen(copia));
}
```

Código-fonte 15 – Exemplo strings e funções `strcpy()`, `strcat()`, `strcmp()` e `strlen()`
Fonte: Elaborado pelo autor (2020)



```
todos = -
copia = -
digite um nome ou fim para encerrar:linguagem

todos = - linguagem
copia = linguagem
digite um nome ou fim para encerrar:C

todos = - linguagemC
copia = C
digite um nome ou fim para encerrar:fim

todos = - linguagemC
comprimento de todos = 12
copia = C
comprimento de copia = 1
Pressione qualquer tecla para continuar. . .
```

Figura 8 –Saída do Código-fonte Exemplo strings e funções strcpy(), strcat(),strmpi() e strlen()
 Fonte: Elaborado pelo autor (2020)

Uma aplicação mais comum seria não apenas ter uma variável que possa conter uma palavra (string), mas armazenar um conjunto de palavras, por exemplo, armazenar os nomes dos alunos de uma sala de aula. Para isso, é preciso trabalhar com um **vetor de string**.

Um **vetor de string** é um vetor que armazena string de 30 caracteres como elemento de um vetor. Supondo uma sala com apenas 4 alunos, deve ser usada a variável nome.

30 caracteres	
0	"Joao da Silva"
1	"Pedro Alvares"
2	"Maria Pedroso"
3	"Ana Souza"

Tabela 4 – Representação de uma variável do tipo vetor de string
 Fonte: Elaborada pelo autor (2020)

O programa abaixo apresenta como declarar e usar a variável nome apresentada na figura anterior.

```
#include <stdio.h>
#include <string.h>

#define N_ALUNOS 4
#define N_CARACTERES 30

int main()
{
    char nome[N_ALUNOS][N_CARACTERES];
    int i;
    for (i=0; i<N_ALUNOS ; i++){
        printf("digite o nome do aluno: ");
        fflush(stdin);
        gets(nome[i]);
    }
}
```

Código-fonte 16 – Exemplo de programa com vetor de strings (nomes)
 Fonte: Elaborado pelo autor (2020)

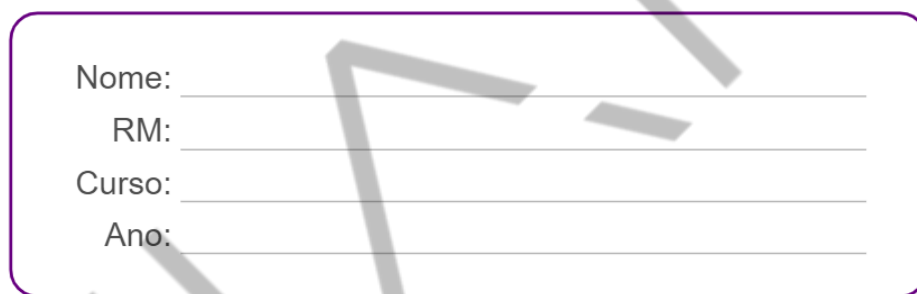
3.3 Registros: estruturas de dados heterogêneas (struct)

Uma estrutura de dados é um conjunto de uma ou mais variáveis, possivelmente de tipos diferentes, colocadas juntas com um único nome. Em algoritmos, esse tipo de variável é chamado de **registro**. Cada informação contida em uma estrutura é denominada de **campo** ou membro.

Por ser composto de informações que podem ser de vários tipos, um registro (uma variável *struct* em linguagem C) é classificado como uma estrutura de dados heterogênea.

Por exemplo, o registro de um aluno poderia ser:

Exemplo registro de um aluno



Nome: _____
RM: _____
Curso: _____
Ano: _____

Figura 9 – Representação de uma variável contendo um registro de um aluno
Fonte: Elaborada pelo autor (2020)

Supondo que seja desejado utilizar no algoritmo um registro que armazene informações referentes a um caderno. Para uma papelaria, seria necessário que o registro do caderno na loja possuía os seguintes campos: fabricante, número de folhas, tipo de capa, tamanho e preço. Já para um estudante muito organizado, o caderno deveria possuir os seguintes campos: curso, ano, disciplina e professor. Assim, deve ficar claro que não há uma única definição de como cada objeto do mundo real deve ser modelado para ser usado na programação de uma aplicação.

A Figura 10 “Tipos diferentes de campos de registro para representar um caderno”, apresenta o que seriam os registros do caderno nas duas situações citadas anteriormente.

Registro Caderno_Papelaria

- Fabricante
- Número de folhas
- Tipo de capa
- Tamanho
- Preço

Registro Caderno_Aluno

- Curso
- Ano
- Disciplina
- Professor

Figura 10 – Tipos diferentes de campos de registro para representar um caderno
Fonte: Elaborada pelo autor (2020)

O próprio registro caderno pode ser usado por momentos e motivos diferentes durante a execução da aplicação. Por exemplo, a variável caderno a ser cadastrado na loja e outra variável para ler o caderno que vai ser comercializado pela loja.

Desta forma, deve-se separar o molde (ou seja, os campos que formarão o registro) da declaração das variáveis que seguirão o molde definido.

Utilizando-se um pseudocódigo tem-se a seguinte estrutura:

```
Algoritmo Papelaria;  
  
Declare tipo  
    Registro Caderno_Papelaria  
    Início  
        fabricante[15]: caractere;  
        num_folhas: inteiro;  
        tipo_de_capa[12]: caractere;  
        tamanho[10]: caractere;  
        preço: real;  
    fim;  
  
    declare variáveis  
        caderno_cadastro: Registro Caderno_Papelaria  
        caderno_venda: Registro Caderno_Papelaria;  
    ...
```

Note que a primeira tarefa a ser realizada é a especificação do molde, ou seja, definição do novo tipo de dado Registro Caderno_Papelaria. Depois podem ser declaradas várias variáveis (no exemplo, as variáveis caderno_cadastro e

caderno_venda) que são do tipo Registro Caderno_Papelaria. A linguagem C segue o mesmo conceito. Em C, o tipo de dado registro é implementado como struct. O programa exemplo, a seguir, apresenta a forma como é definido o molde e também como são declaradas as variáveis. Também mostra como se tem acesso a cada campo que compõe a variável do tipo struct.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct Caderno_Papelaria {
    char fabricante[15];
    int num_folhas;
    char tipo_de_capa[12];
    char tamanho[10];
    float preco;
};

int main(){
    /* declaração das variáveis */
    struct Caderno_Papelaria caderno_cadastro, caderno_venda;

    /*Leitura de uma variável do tipo struct: o caderno a ser
    cadastrado*/
    printf( "digite o caderno a ser cadastrado: \n");
    printf( "digite o fabricante: ");
    fflush(stdin);
    gets(caderno_cadastro.fabricante);
    printf( "digite o numero de folhas: ");
    scanf("%d", &caderno_cadastro.num_folhas);
    printf( "digite o tipo de capa: (dura/brochura)");
    fflush(stdin);
    gets(caderno_cadastro.tipo_de_capa);
    printf( "digite a dimensao: ");
    fflush(stdin);
    gets(caderno_cadastro.tamanho);
    printf( "digite o preco: ");
    scanf ("%f", &caderno_cadastro.preco);

    /*Supondo que o caderno cadastrado seja vendido*/
    caderno_venda = caderno_cadastro;

    /*Escrita das informacoes do caderno que é vendido*/
    printf( "\n\nApresentacao do caderno vendido \n");
    printf("fabricante:%s\n", caderno_cadastro.fabricante);
    printf("num. de folhas: %d \n",
    caderno_venda.num_folhas);
    printf( "tipo de capa: %s
    \n",caderno_venda.tipo_de_capa);
    printf( "dimensao: %s \n",caderno_venda.tamanho);
    printf( "preco: R$%0.2f \n", caderno_venda.preco);
}
```

Código-fonte 17 – Exemplo de programa para declaração de struct e seu uso (struct caderno)

Fonte: Elaborado pelo autor (2020)

No exemplo anterior é possível verificar que para se ter acesso a cada campo de uma variável do tipo *struct* é necessário que cada campo seja especificado, e a forma é sempre: *nome_variavel.campo*. Por exemplo, *caderno_cadastro.preco*, onde *caderno_cadastro* é o nome da variável e *preco* é o nome do campo que está sendo acessado. Para realizar a leitura de uma variável do tipo *struct*, campo por campo deve ser lido para poder armazenar as informações desejadas. O mesmo ocorre com a escrita de cada campo da variável.

Já para uma variável que é do mesmo tipo (ou seja, *struct Caderno_Papelaria*), a atribuição de cada campo pode ser feita de forma implícita, é o que ocorre no programa quando: *caderno_venda=caderno_cadastro*. A seguir, outro exemplo de programa que declara e utiliza um tipo de dado *struct*.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct rotulo{
    int x;
    float y;
    char n[20];
};

int main()
{
    struct rotulo qlq = {1,2.3,"Primeiro teste"};
    //declara uma variável struct e atribui valores iniciais
    printf(" x= %d \t y= %f \t n= %s \n", qlq.x ,qlq.y
,qlq.n);
    qlq.x=0;
    strcpy(qlq.n,"segundo");
    printf(" x= %d \t y= %f \t n= %s \n", qlq.x ,qlq.y
,qlq.n);
}
```

Código-fonte 18 – Exemplo 2 de programa para declaração de struct e sua manipulação

Fonte: Elaborado pelo autor (2020)

O uso de *struct* é realmente mais utilizado quando podem ser armazenados vários registros de um determinado objeto, assim os exemplos que se seguem são referentes ao uso de *structs* como elementos de vetores ou matrizes.

3.4 Uso de vetor de struct

Um vetor de elementos do tipo *struct* deve ser declarado da seguinte forma:

```
struct molde vetor[N];
```

Isso significa que cada elemento que forma a variável vetor será uma *struct* com todos os campos especificados pelo molde.

Para entender melhor, vamos ver outros exemplos de uso de *struct* como elementos de vetores.

Suponha que um programa deva cadastrar as informações (nome e nota) de 3 alunos de uma sala de aula e apresentar as informações do aluno com maior nota. O programa cria uma variável chamada *aluno* que pode ser representada conforme a figura “Representação de um vetor de registros de alunos”.

Nome:	MARIA
RM:	4567
Curso:	SI
Ano:	2017

Nome:	PEDRO
RM:	7890
Curso:	ENGENHARIA
Ano:	2017

Nome:	JOÃO
RM:	1234
Curso:	SI
Ano:	2015

Figura 11 – Representação de um vetor de registros de alunos
Fonte: Elaborado pelo autor (2020)

O programa apresentado pelo código-fonte a seguir cria o tipo de dado *struct* cadastro e declara a variável *aluno* com 3 elementos do tipo *struct*.

```
#include <stdlib.h>
#include <stdio.h>
#include <string.h>

struct cadastro{ //cria o tipo de dado struct aluno
    char nome[10];
    float nota;
};
```

```
int main()
{
    struct cadastro aluno[3]; /*declara variável aluno
                               como vetor de 3 elementos do tipo struct aluno*/
    struct aluno aluno_maior;
    /*declara variável aluno_maior que armazenará
       as informações do(1)aluno com maior nota */
    float maior, nota;
    int i;

    /* cadastra 3 alunos e suas notas*/
    for(i=0;i<3;i++){
        printf("digite nome: ");
        fflush(stdin);
        gets(aluno[i].nome);
        printf("digite nota: ");
        scanf("%f", &aluno[i].nota);
    }

    /* Procura e apresenta o aluno com maior nota*/
    maior = 0;
    for(i=0;i<3;i++){
        if(aluno[i].nota > maior){
            maior = aluno[i].nota;
            aluno_maior.nota=maior;
            strcpy(aluno_maior.nome,aluno[i].nome);
        }
    }
    printf("\n*****\n");
    printf("\t maior nota do aluno = %s \t nota = %0.1f \n\n",
    aluno_maior.nome,  aluno_maior.nota);
}
```

Código-fonte 19 – Programa exemplo de vetor com registros (struct) de 3 alunos
Fonte: Elaborado pelo autor (2020)

O uso de struct em C é bastante extenso sendo que cada campo que compõe um registro pode ser não apenas de tipo simples de dados, mas, também, tipo composto, por exemplo, vetor, matriz e inclusive registro.

3.5 Variáveis do Tipo Ponteiro

Há duas situações em que se pode utilizar variáveis do tipo ponteiros:

1. A passagem de parâmetros para funções por referência;
2. A alocação dinâmica de variáveis na memória RAM.

Esses dois assuntos ainda serão abordados, por esse motivo é preciso entender primeiro o que são variáveis do tipo ponteiro e como devemos manipulá-las para depois fazer uso deste conhecimento.

Quando uma variável é declarada, um espaço na memória RAM é reservado para ela. Lembre-se que cada posição de memória é identificada com um endereço único. Imagine que quando a variável *x* foi declarada, o endereço escolhido para a variável *x* seja o endereço 2500. Em programação é permitido o acesso a uma variável pelo nome ou símbolo que foi criado, sendo assim, quando o valor 3 é atribuído à variável *x* (*x*=3), o conteúdo de memória recebe o valor 3. A Figura 12 “Representação de uma variável na memória RAM” mostra o resultado da declaração e atribuição.

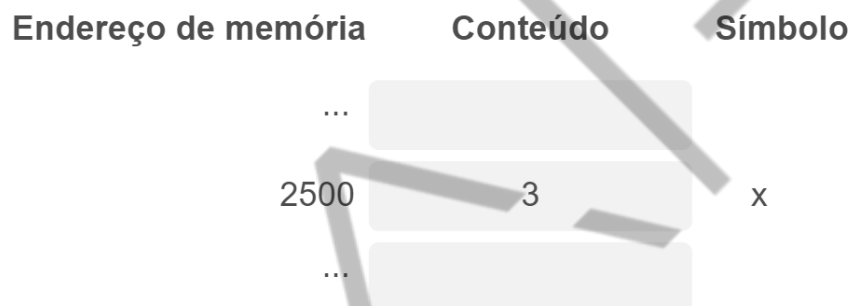


Figura 12 – Representação de uma variável na memória RAM
Fonte: Elaborado pelo autor (2020)

Definição de Ponteiro: uma variável é dita do tipo ponteiro se ela contiver como informação um endereço de memória RAM.

IMPORTANTE: vale ressaltar que ponteiro é uma variável que aponta para uma posição da memória RAM.

Para declarar e manipular variáveis do tipo ponteiro são necessários dois operadores:

1. **&** (endereço de): que retorna o endereço de memória de uma variável.
2. ***** (conteúdo de): que especifica que se deve manipular o conteúdo de um endereço de memória.

Exemplo de declaração de variável do tipo ponteiro:

```
int *ptr;
```

Toda variável do tipo ponteiro tem o operador * antes do seu nome. A declaração deve ser lida da seguinte forma: ptr é uma variável do tipo ponteiro cujo conteúdo do que ela aponta é inteiro.

A explicação fica mais fácil se for analisado o trecho de programa:

```
int x;  
int *ptr; //declara ptr como ponteiro para um dado  
inteiro  
x = 3;  
ptr = &x; /*atribui à variável ptr o endereço de memória  
onde está a variável x, ou seja, ptr aponta  
para área da memória onde está a variável x */
```

A figura “Representação de uma variável ponteiro” mostra o efeito dessa sequência de comandos.



Figura 13 – Representação de uma variável ponteiro
Fonte: Elaborado pelo autor (2020)

Suponha que mais um comando é executado após o trecho anterior:

```
*ptr = 7; //conteúdo do que ptr aponta recebe o valor 7
```

Esse comando deve ser lido da seguinte forma: o conteúdo do que está sendo apontado por ptr recebe o valor 7. Portanto, a variável x apontada por ptr deixa de valer 3 e passar a conter o valor 7, conforme mostra a figura a seguir.



Figura 14 – Representação de uma variável sendo alterada por seu ponteiro
Fonte: Elaborado pelo autor (2020)

Um programa completo, apresentado as etapas aqui descritas é:

```
#include <stdio.h>
int main()
{
    int x = 3;
    int *ptr;
    ptr = &x;
    printf(" x=%d ptr=%d conteudo do que ptr aponta %d \n",x,
    ptr, *ptr);
    //conteúdo do que ptr aponta recebe o valor 7
    *ptr = 7;
    printf(" x=%d ptr=%d conteudo do que ptr aponta %d \n",x,
    ptr, *ptr);
}
```

Código-fonte 20 – Programa exemplo de declaração e uso de ponteiro para dado inteiro
Fonte: Elaborado pelo autor (2020)

Observe na tela de saída que o valor contido em ptr (que é ponteiro), é o endereço de memória reservada para a variável x pelo compilador (em cada máquina e execução endereços diferentes podem ser obtidos). Já quando o valor apresentado por *ptr (conteúdo do que ptr aponta) é o conteúdo da variável x, uma vez que ptr está apontado para a variável x (pelo comando ptr = &x).

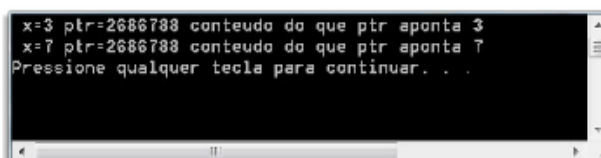


Figura 15 – Tela de saída do Código-fonte Programa exemplo de declaração e uso de ponteiro para dado inteiro
Fonte: Elaborado pelo autor (2020)

No próximo código são feitas as mesmas operações com ponteiro, com uma diferença de que o dado onde o ponteiro aponta é do tipo float.

```
#include <stdio.h>
int main()
{
    float x;
    float *pontx;
    pontx = &x;    //pontx passa apontar para a variável x
    *pontx = 3.14; //conteúdo do que pontx aponta recebe 3.14
    printf(" x=%0.2f pontx=%d conteudo do que pontx aponta %f\n",x, pontx, *pontx);
}
```

Código-fonte 21 – Programa exemplo de declaração de uso e ponteiro para dado float
Fonte: Elaborado pelo autor (2020)

Para concluir a apresentação feita sobre o tipo de dado ponteiro e sua manipulação, falta apenas conhecer como são tratados os ponteiros para o tipo de dado *struct*. E para tal, será utilizado o programa a seguir, que segue a mesma sequência de comandos do programa anterior, mas desta vez usando um ponteiro para uma variável do tipo *struct*.

```
#include <stdio.h>

struct cad{
    int RM;
    char nome[30];
};

int main()
{
    struct cad aluno; //declara variável do tipo struct cad
    struct cad *p; //declara ponteiro para struct cad
    p = &aluno; //p aponta para a variável aluno
    printf(" digite RM do aluno: ");
    //lê valor para o campo RM da variavel apontada por p
    scanf("%d",&p->RM);
    printf(" digite nome do aluno: ");
    fflush(stdin);
    //lê valor para o campo nome da variavel apontada por p
    gets(p->nome);
    printf(" digite RM do aluno: ");
    scanf("%d",&p->RM);
    printf(" O aluno %s tem RM:%d \n",p->nome, p->RM);
}
```

Código-fonte 22 – Programa exemplo de declaração de uso de ponteiro para dado struct
Fonte: Elaborado pelo autor (2020)

São declaradas duas variáveis: *aluno* do tipo *struct cad* e *p* um ponteiro para *struct cad*. Depois do comando *p=&aluno*, o ponteiro *p* passa a apontar para a área reservada para a variável *aluno* na memória, como mostra a figura.

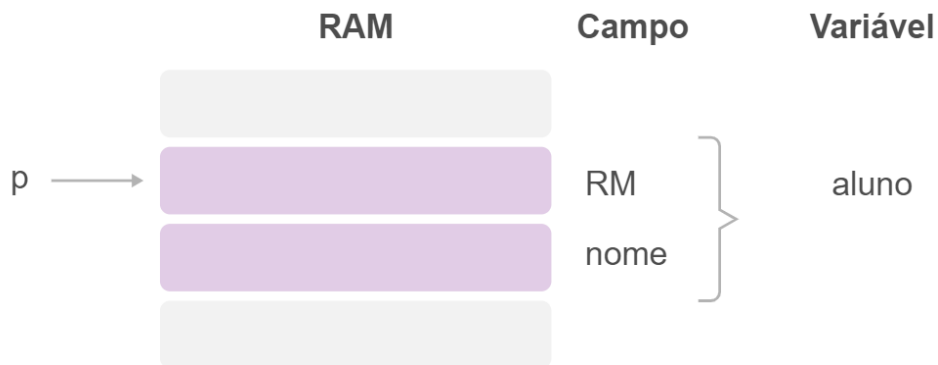


Figura 16 – Exemplo de esquema de ponteiro apontando para uma variável do tipo struct
 Fonte: Elaborado pelo autor (2020)

Para ter acesso aos campos da variável aluno através do ponteiro p é preciso conhecer mais um operador em C: `->`. E quando se encontra no código `p->` deve-se ler da seguinte forma: “selecione a struct apontada por p”. Tendo acessado à variável do tipo struct cad é preciso selecionar o campo dentro da struct cad, o que acontece com `p->RM` e `p->nome`.

4 FUNÇÕES

Uma função é um conjunto de comandos agrupados em um **módulo** ou **bloco**, que recebe um **nome** e através deste nome pode ser **chamado** para ser executado.

Em todas as linguagens é necessário haver construções que permitam escrever programas usando o conceito de modularização. As principais razões para utilizar funções na programação de softwares são:

- Para permitir o reaproveitamento de código já construído;
- Para evitar que um trecho de código seja repetido várias vezes dentro de um mesmo programa;
- Para permitir a alteração de um trecho de código de uma forma mais rápida. Com o uso de uma função é preciso alterar apenas dentro da função que se deseja;
- Para que os blocos do programa não fiquem grandes demais e, por consequência, mais difíceis de entender;
- Para facilitar a leitura do programa-fonte de uma forma mais fácil;

- Para separar o programa em partes (blocos) que possam ser logicamente compreendidos de forma isolada.

Assim, é preciso observar a definição da estrutura básica de uma função em C, a qual é apresentada no Código-fonte “Estrutura básica de uma função em C”.

```
<tipo do valor retornado> nome da função(lista de
parâmetros)
{
//declaração de variáveis locais
//lista de comandos
return (valor único retornado);
}
```

Código-fonte 23 – Estrutura básica de uma função em C
Fonte: Elaborado pelo autor (2020)

Toda função tem que ter um nome e dentro de delimitadores (“{” e “}”) a especificação dos comandos a serem executados quando a função for chamada.

As variáveis declaradas dentro da função são as variáveis locais, ou seja, só são conhecidas, e consequentemente utilizadas, enquanto a função estiver sendo executada.

Toda informação que deve ser passada para a função oriunda da função que chama essa função para ser executada, bem como valores gerados como resultados da execução da função, são devolvidos para quem chamou essa função e são passados como parâmetros. Mesmo que não haja parâmetros a frente do nome da função deve-se ter parênteses ().

4.1 Funções com apenas 1 valor de resultado (return)

Em C, uma função pode retornar 1 valor com o próprio nome da função, ou seja, usando o comando return. No caso de retornar um valor de resultado desta forma, à frente do nome da função deve ser colocado o tipo do valor retornado, por exemplo, int, float, char etc. O retorno do valor do tipo especificado é feito através do comando return.

Quando nenhum valor é retornado com o nome da função, deve ser colocado o tipo void, que significa nenhum valor será retornado.

Por exemplo, veja como desenvolver um programa que calcule (usando uma função) o discriminante de uma equação de segundo grau do tipo $ax^2 + bx + c = 0$. Lembrando que o delta (discriminante) é $b^2 - 4ac$. Assim, a função delta exigiria a seguinte descrição da troca de parâmetros:

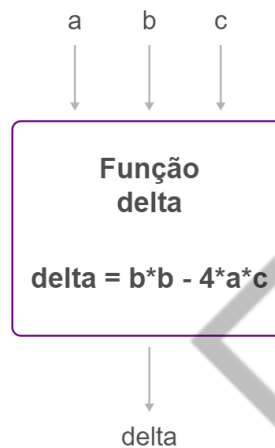


Figura 17 – Representação de função delta usada no programa exemplo
Fonte: Elaborado pelo autor (2020)

Os parâmetros a, b e c são chamados parâmetros de entrada, pois são valores que a função precisa receber para poder calcular o delta. O delta calculado é um parâmetro de saída.

Nesse primeiro momento, pode-se utilizar apenas uma forma de retornar o resultado que será usando o comando return, conforme pode ser visto no programa a seguir.

```
#include <stdio.h>
#include <stdlib.h>

//função p/ calcular o discriminante de uma equação do 2º
grau
int delta(int a,int b, int c)
{
    int d;
    d= b*b-4*a*c;
    return(d);
}
int main()
{
    int x,y,z,d;
    printf("digite 3 coeficientes:");
    scanf("%d%d%d", &x,&y,&z);
    d=delta(x,y,z);
    printf ("discriminante: %d \n", d);
}
```

```
}

```

Código-fonte 24 – Programa com função que calcula o discriminante de equação do 2º grau
Fonte: Elaborado pelo autor (2020)

A especificação das funções que fazem parte do programa deve acontecer antes que a função seja chamada, por isso a função delta é codificada antes da função main(). E a execução de qualquer programa é iniciada sempre na função main(), sendo assim, a execução inicia declarando as variáveis locais da função main(). Dessa forma, apenas a função main() está ativa na memória, existindo apenas as variáveis x, y, z, e d.

A Figura “Representação da função main antes da chamada da função delta” apresenta uma representação do que é a função main() até o momento em que a chamada irá ocorrer, já supondo que a leitura dos valores de x, y e z já tenha sido feita (por isso estão com cor branca).

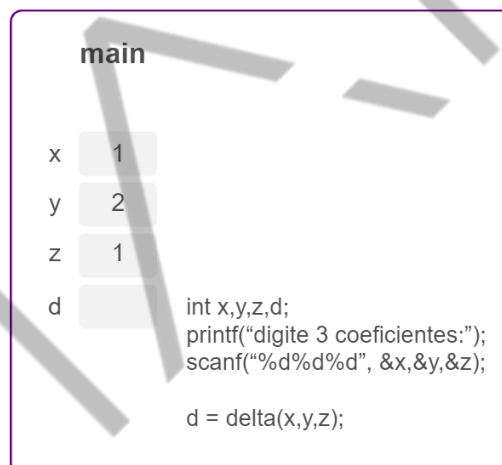


Figura 18 – Representação da função main antes da chamada da função delta
Fonte: Elaborado pelo autor (2020)

Quando é feita a chamada da função delta(), a função main() fica “inativa” na memória e passa apenas a ser executada a função delta(). Porém, antes da função main() ficar aguardando o resultado da função delta são passados os parâmetros necessários para a função delta poder ser executada. Os parâmetros são relacionados com as posições em que são declarados na função delta e a ordem em que as variáveis da função main() são colocadas.

No exemplo, o parâmetro a recebe o valor da variável x, b o valor da variável y e c o valor da variável z. Portanto, a = 1, b = 2 e c = 1. Após a passagem dos

parâmetros, a função delta inicia sua execução e como primeiro comando é feita a declaração da variável local d.

A figura “Representação da função delta quando inicia sua execução delta” mostra como a função delta recebe os parâmetros de entrada e usa a variável local d.

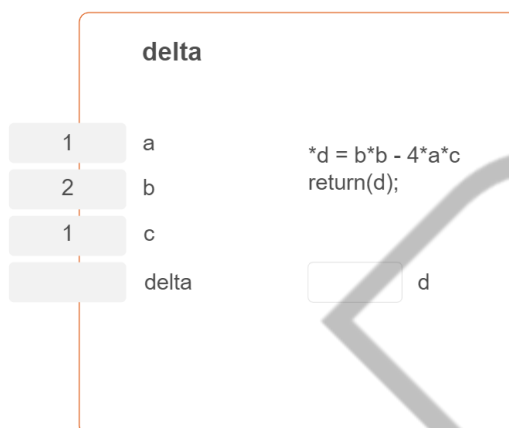


Figura 19 – Representação da função delta quando inicia sua execução delta
Fonte: Elaborado pelo autor (2020)

É muito importante notar que apesar de ter o mesmo nome da variável d da função main() não há qualquer relação entre elas, pois a variável d da função delta() apenas existe enquanto a função delta estiver sendo executada.

Em seguida, o cálculo é realizado e armazenado na variável local d. Para que esse resultado possa ser enviado para a função main() é usado o comando return(d). Esse comando faz com que o valor do resultado seja devolvido para a função que fez a chamada da função delta() e, em seguida, o comando return encerra a execução da função delta(). Quando o comando return é executado, a função que fez a chamada deve ter uma variável que receba através de uma atribuição o valor retornado da função.

Note que o tipo do retorno da função delta() é int e a variável d da função main() foi declarada como int. Assim, como está sendo atribuído a variável d o resultado da função, essa variável guarda o valor que foi devolvido à função main() pelo return da função delta().

IMPORTANTE: é fundamental saber que o return é capaz apenas de retornar 1 valor que pode ser do tipo int, float, char, struct ou ponteiro (a ser estudado posteriormente). Claro que se deve atribuir a uma variável que seja do mesmo tipo do valor declarado do retorno da função.

Apenas para ficar mais clara a utilização das chamadas de função retornando resultados com o comando return, observe mais um exemplo:

```
#include <stdio.h>

char avaliacao (float peso1, float peso2)
{
    if (peso1 == peso2)
        return 'p'; //permanece igual
    else if (peso1 > peso2)
        return 'd'; //diminuiu o peso
    else
        return 'a'; //aumentou o peso
}

int main()
{
    char situacao;
    float peso_inicial, peso_final;
    printf("Digite peso antes e depois da dieta");
    scanf("%f%f", &peso_inicial, &peso_final);
    situacao = avaliacao (peso_inicial, peso_final);
    if (situacao == 'i')
        printf("O paciente não emagreceu\n");
    else if (situacao == 'd')
        printf("O paciente emagreceu\n");
    else
        printf("O paciente engordou\n");
}
```

Código-fonte 25 – Programa com função que verifica situação de peso
Fonte: Elaborado pelo autor (2020)

4.2 Funções com vários resultados (parâmetros de saída)

Até agora, todas as funções apresentadas podiam:

1. Receber vários parâmetros como valores de entrada para a função.
2. Apenas gerava 1 valor como resultado da função.

E se for necessário gerar mais de um resultado? São três os possíveis tipos de parâmetros de uma função, como mostra a figura “Representação de parâmetros de entrada e saída de uma função”:

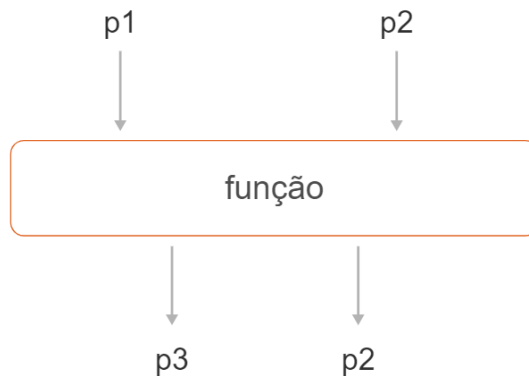


Figura 20 – Representação de parâmetros de entrada e saída de uma função

Fonte: Produzido pelo autor (2020)

1. Entrada: valores apenas para serem usados pela função (p1).
2. Entrada e Saída: são aqueles cujos valores são usados dentro da função e que sofrem alterações durante a execução da função e precisam ser passados para a função que fez a chamada (p2).
3. Saída: são resultados que a função gera e que precisam ser passados para a função que fez a chamada (p3).

Em C, os parâmetros de uma função podem ser passados:

1. Por valor:

Um valor é copiado para o parâmetro da função e pode ser usado pela função. Apenas para parâmetros de entrada da função (no exemplo, p1).

2. Por referência (ponteiro):

O parâmetro é uma ligação (uma referência) a uma variável da função que fez a chamada, assim toda alteração que a função impor ao parâmetro terá efeito na variável ligada ao parâmetro. Usado para parâmetros de Saída e Entrada e Saída (no exemplo, p2 e p3).

Resumindo na figura:

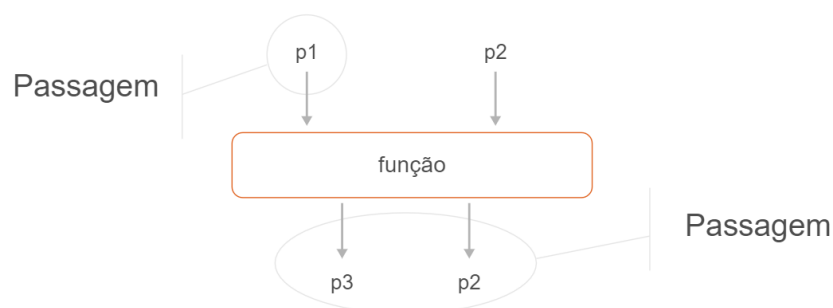


Figura 21 – Passagem de parâmetros de entrada (por valor) e saída (por referência)
 Fonte: Elaborado pelo autor (2020)

Suponha a função exemplo com parâmetros definidos na Figura 22. Apenas analisando a figura podemos escrever o cabeçalho (protótipo) da função exemplo. Nessa implementação não vamos usar o comando return para devolver um resultado da função para quem fez a chamada assim, o tipo de retorno da função é void. O parâmetro n (inteiro) é apenas de entrada, ou seja, passagem por valor. Finalmente, v (float) e res (inteiro) são parâmetros de saída e entrada e saída, respectivamente. Sendo assim, devem ser passados por referência, ou melhor, como ponteiros.

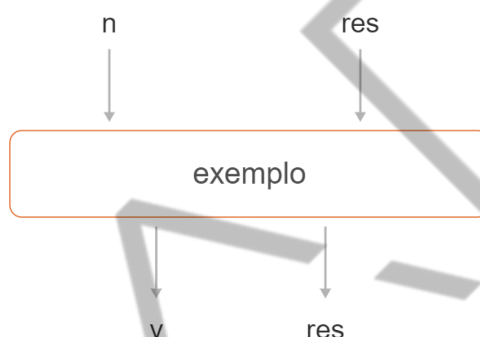


Figura 22 – Representação de parâmetros de entrada e saída de uma função exemplo
 Fonte: Elaborado pelo autor (2020)

Assim, o protótipo da função exemplo é:

```
void exemplo (int n, int *res, float *v)
```

O programa inteiro que faz uso da função exemplo é apresentado.

```
#include <stdio.h>

void exemplo (int n, int *res, float *v)
{
    n++;
    *res = n + *res;
    *v = n * 2.1;
}

int main()
{
    int a = 1, d = 2;
    float m = 0;
    exemplo (a, &d, &m);
    printf("a= %d \t y = %d\t e m = %.1f\n", a, d, m);
}
```

Código-fonte 26 – Função com passagem de parâmetros por valor e por referência.
 Fonte: Elaborado pelo autor (2020)

Para realizar a chamada da função exemplo() são passados os parâmetros, sendo que a variável a tem seu valor passado para o parâmetro de entrada n. Já os parâmetros res e v da função exemplo são parâmetros saída, ou seja, devem ser ponteiros. Lembrando da definição de ponteiro que diz que uma variável do tipo ponteiro é aquela que sempre armazena endereço de memória, ao invés de passar o valor de d e m para res e v devem ser passados os seus endereços usando o operador &.

Considere a figura seguinte, que mostra uma forma de representar o que acontece no momento da chamada da função e passagem de parâmetros. Note que o parâmetro res é um ponteiro que aponta para a variável d e o parâmetro v é um ponteiro que aponta para a variável m da função main().

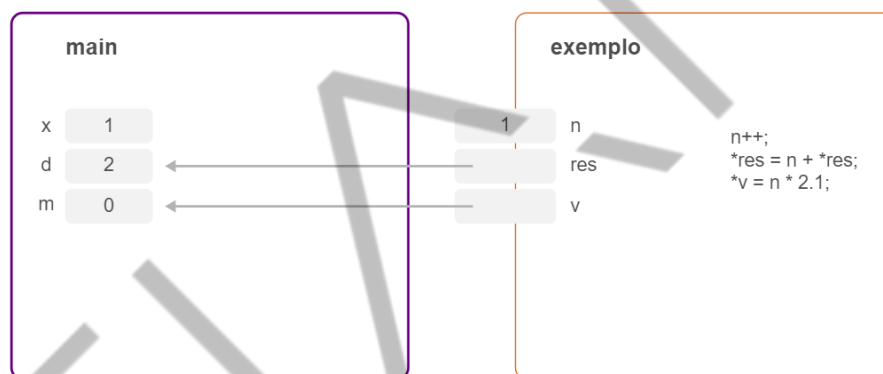


Figura 23 – Esquema da passagem de parâmetros para função exemplo
Fonte: Elaborado pelo autor (2020)

Descrevendo a execução da função exemplo():

1. No comando `n++`, ação se dá sobre o valor armazenado em `n` na passagem por valor, que foi o valor 1. Assim, `n` termina com o valor 2.
2. O valor de `n` é somado ao conteúdo de memória apontado por `res`, ou seja, o valor 2 armazenado na variável `d` da função `main()`. O resultado 4 é armazenado como conteúdo da variável apontado por `res`.
3. Finalmente, a multiplicação realizada gera como resultado 8.2 e deve ser armazenado como conteúdo da variável apontada pelo parâmetro `v`, isto é, na variável `m` da função `main()`.

A Figura 24 “Esquema do retorno da passagem de parâmetros para função exemplo” mostra como fica o esquema de passagem de parâmetros e as variáveis da função main() após todos os comandos da execução da função exemplo().

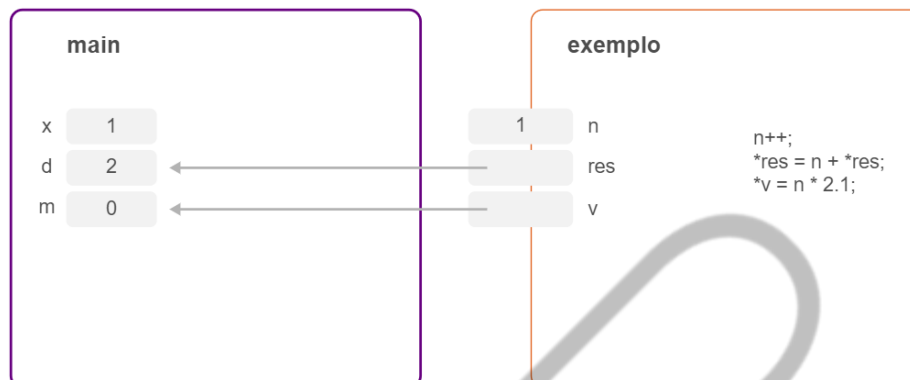


Figura 24 – Esquema do retorno da passagem de parâmetros para função exemplo
Fonte: Elaborado pelo autor (2020)

É possível modificar a função que calcula o delta da função de 2º grau que programamos usando o comando `return` apresentada na Figura “Representação de função delta usada no programa exemplo”, e para não usar o comando `return` o resultado deve ser retornado usando passagem de parâmetro por referência (ponteiro). Veja, a nova implementação fica como mostra o código “Programa com função delta com passagem de parâmetros por valor e por referência”.

```
#include <stdio.h>

//função para calcular o discriminante da equação do 2º
grau
void delta(int a,int b, int c, int *d)
{
    *d= b*b-4*a*c;
}

int main()
{
    int x,y,z,d;
    printf("digite 3 coeficientes:");
    scanf("%d%d%d", &x,&y,&z);
    delta(x, y, z, &d);
    printf ("discriminante: %d \n", d);
}
```

Código-fonte 27 – Função delta com passagem de parâmetros por valor e por referência
Fonte: Elaborado pelo autor (2020)

Observe que, como o comando `return` não é usado, o tipo da função foi alterado para `void` e o quarto parâmetro foi incluído para retornar o resultado da função para a função `main()`.

A figura a seguir apresenta a passagem de parâmetros realizada usando o programa anterior.

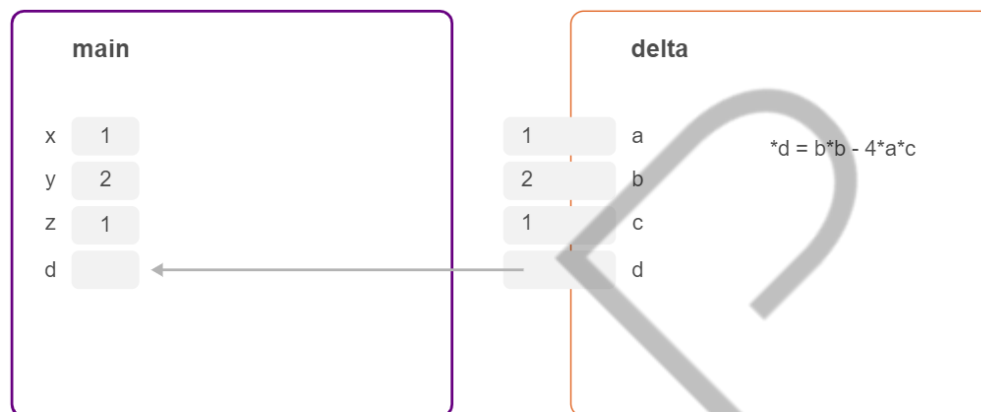


Figura 25 – Esquema da passagem de parâmetros por valor e por referência para função `delta`
Fonte: Elaborado pelo autor (2020)

Os parâmetros `a`, `b` e `c` continuam recebendo os valores (passagem de parâmetro por valor) das variáveis `x`, `y` e `z` da função `main()`. Já o parâmetro `d` é um ponteiro que recebe o endereço da variável `d` da função `main()`. Desta forma, é representado na figura como uma seta (ponteiro) que aponta para a variável `d` da função `main()`.

Após a execução do cálculo $(b*b - 4*a*c)$ dentro da função `delta`, o resultado é armazenado como conteúdo da variável apontada pelo parâmetro `d` (`*d`), que será na variável `d` da função `main()`. Portanto, a alteração do parâmetro `d` dentro da função `delta` na verdade altera diretamente a variável `d` da função `main()`.

4.3 Funções com passagem de parâmetros do tipo de dado struct

A regra é a mesma para variáveis como `int` ou `float`: se um dado do tipo struct é parâmetro de saída de uma função, este deve ser passado por referência, ou seja, usando ponteiro.

A ideia é a de utilizar um programa que possui 2 funções:

1. A função `le_registro()` deve ler do teclado os campos que compõe a struct `prod` devolvendo para a função que a chamou um parâmetro com a struct

prod alterada. Desta forma, tem apenas uma struct prod como parâmetro de saída.

2. Já a função escreve() recebe como parâmetro uma struct prod e deve apenas escrever na tela de saída sem gerar qualquer alteração no conteúdo dos campos. Portanto, recebe apenas um parâmetro de entrada.

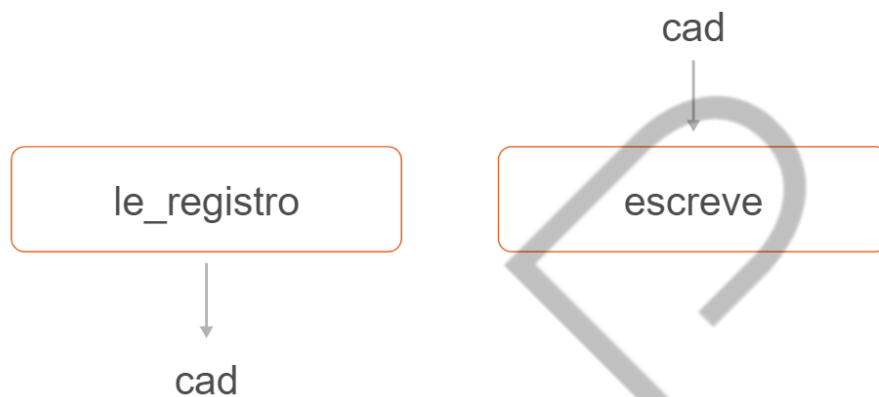


Figura 26 – Representação passagem de parâmetros das funções le_registro() e escreve()
 Fonte: Elaborado pelo autor (2020)

```

#include<stdio.h>
#include<string.h>

struct prod{
    int cod;
    float preco;
};

//parametro de saida p_cad é ponteiro para struct prod
void le_registro(struct prod *p_cad)
{
    printf("Digite codigo do produto e preco\n");
    scanf("%d", &p_cad->cod);
    scanf("%f", &p_cad->preco);
}

//parametro de entrada cad é uma struct prod
void escreve(struct prod reg) {
    printf("Codigo e preco do produto sao: %d R$%.2f \n",
    reg.cod, reg.preco);
}

int main() {
    struct prod cad;
    //chama função passando o endereço da variável cad
    le_registro (&cad);
    //chama função passando o valor da variável cad
    escreve(cad);
}
  
```

Código-fonte 28 – Programa com funções com passagem de parâmetros do tipo struct
Fonte: Elaborado pelo autor (2020)

Na função `main()`, é feita a declaração da variável **cad** que é do tipo *struct prod*.

Em seguida, é feita a chamada da função `le_registro(&cad)`. Assim, pela passagem de parâmetro **p_cad** (do tipo ponteiro para uma *struct prod*) recebe o endereço de memória (ou seja, aponta) onde se encontra a variável **cad** (`p_cad = &cad`).

Dentro da função `le_registro()`:

3. `p_cad->cod` especifica que o campo `cod` da struct apontada por `p_cad` (deve ser lido como “o campo `cod` da struct apontada por `p_cad`”).
4. `p_cad->preco` especifica que o campo `preco` da struct apontada por `p_cad` (deve ser lido como “o campo `preco` da struct apontada por `p_cad`”).

Quando a função `le_registro()` termina a variável `cad` da função `main()`, ela tem os valores dos campos lidos dentro da função.

O parâmetro de entrada `reg` da função `escreve()` recebe uma cópia da variável `cad` da função `main()`. Desta forma, a parâmetro do tipo struct *prod* `reg` tem seus campos escritos na tela de saída de forma normal.

4.4 Vetores, string e matrizes como parâmetros de funções

Todos os parâmetros que são de saída, ou de entrada e saída devem ser passados por referência, ou seja, como ponteiros. Com vetores e matrizes também não é diferente, assim deve-se conhecer como passá-los como ponteiro.

Vetores e matrizes seguem o mesmo conceito para serem passados como parâmetros por referência. Em C, quando é declarado um vetor ou uma matriz é reservada em uma área de memória para o armazenamento dos elementos. Quando se referencia apenas o nome do vetor é obtido o endereço inicial onde foi armazenado o 1º elemento da estrutura.

endereço	conteúdo	identificador ou nome da variável
1000	'b'	vetor[0]
1001	'd'	vetor[1]
1002	'x'	vetor[2]

```

char vetor[3] = {'b', 'd', 'x'};
char *pont
pont = vetor; /* equivalente à pont = &vetor[0] */

```

Figura 27 – Representação declaração de vetor e de ponteiro pelo nome do vetor
Fonte: Elaborado pelo autor (2020)

O ponteiro `pont` terá como conteúdo o endereço do 1º elemento do vetor. E a mesma explicação vale para matrizes, ou seja, estruturas de dados como vetores e matrizes têm a mesma forma de serem passadas como parâmetros de funções. Desta forma, para passar um vetor ou matriz como parâmetro, basta usar o seu nome sem índice.

IMPORTANTE: vetores e matrizes são sempre passados por referência de forma implícita (não precisamos usar explicitamente ponteiros).

Os exemplos a seguir apresentam vetores passados como parâmetros em várias situações.

Suponha um programa que leia as quantidades vendidas de cada um dos 10 produtos em uma loja em 2 dias consecutivos. O programa deve apresentar o total vendido de cada produto nos 2 dias usando uma função para somar os elementos dos 2 vetores. A passagem de parâmetros para a função `soma_vetor()` é apresentada na figura “Representação da passagem de parâmetros do tipo vetor para função `soma_vetor`”.

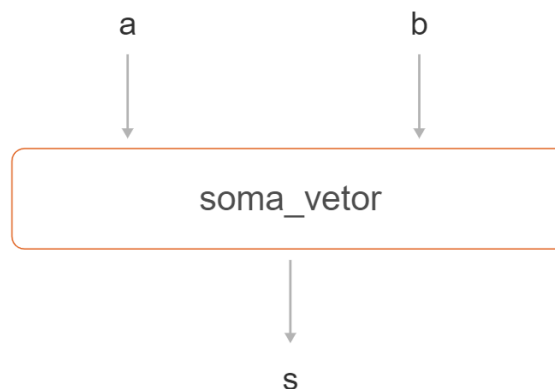


Figura 28 – Representação da passagem de parâmetros do tipo vetor para função `soma_vetor`
Fonte: Elaborado pelo autor (2020)

```
#include <stdio.h>
```

```
#define N 300

/*a e b sao parametros de entrada e s é de saída, mas não
há
diferença para a função */
void soma_vetor(int a[N],int b[N], int s[N])
{
    int i;
    for (i=0; i<N;i++)
        s[i]=a[i]+b[i];
}

int main(){
    int i,venda1[N],venda2[N],total[N];
    for (i=0;i<N;i++){
        printf(" digite quantidade vendida 1o dia do produto %d:
",i);
        scanf("%d",&venda1[i]);
    }
    for (i=0;i<N;i++){
        printf("Digite quantidade vendida 2o dia do produto %d:
",i);
        scanf("%d",&venda2[i]);
    }
    //chama a função passando por referencia cada vetores
    soma_vetor(venda1,venda2,total);
    for (i=0;i<N;i++){
        printf(" A qtd vendida nos 2 dias do produto %d:
%d\n",i,total[i]);
    }
}
```

Código-fonte 29 – Programa com passagem de parâmetros do tipo vetor
Fonte: Elaborado pelo autor (2020)

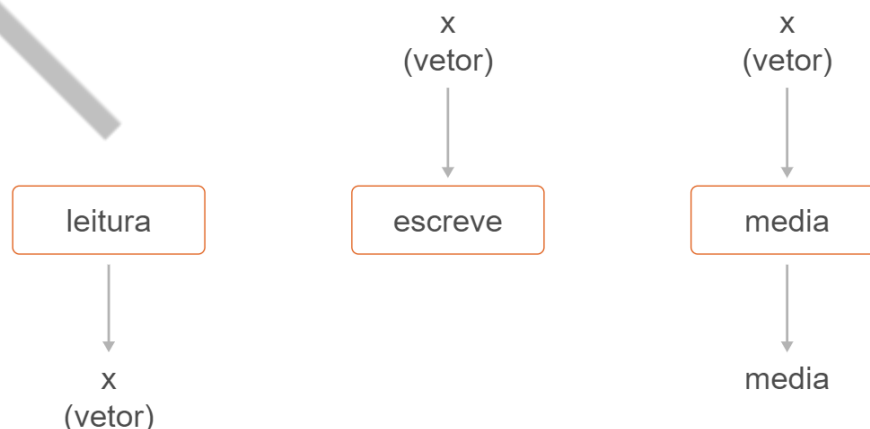


Figura 29 – Representação da passagem de parâmetros para funções
Fonte: Elaborado pelo autor (2020)

Para ter mais um exemplo, o exemplo anterior pode ser modificado da seguinte maneira:

1. Uma função que realize a leitura dos elementos de um vetor com N elementos e a utilize para fazer a entrada de dados dos 2 vetores;
2. Uma função para escrever um vetor com N elementos para escrever o vetor soma;
3. Uma função que calcule a média de produtos vendidos de cada dia.

O esquema de parâmetros destas funções é apresentado na Figura 29 “Representação da passagem de parâmetros para funções”.

```
#include <stdio.h>

#define N 300

void leitura(int x[N])
//vetor x é parametro de saída
{
    int i;
    for (i=0;i<N;i++){
        printf(" digite quantidade vendida do produto %d: ",i);
        scanf("%d",&x[i]);
    }
}

void escreve(int x[N])
//vetor x é parametro de entrada
{
    int i;
    for (i=0;i<N;i++){
        printf(" A quantidade vendida nos 2 dias do produto %d: %d\n",i,x[i]);
    }
}

void soma_vetor(int a[N],int b[N], int s[N])
/*vetores a e b são parâmetros de entrada e vetor s é parâmetro de saída */
{
    int i;
    for (i=0; i<N;i++){
        s[i]=a[i]+b[i];
    }
}

float media(int x[N])
```

```
/*vetor x é parametro de entrada e valor da média
calculada
é retornado pelo comando return */
{
int i,soma=0;
float aux;
for (i=0;i<N;i++)
    soma=soma+x[i];
aux= (float) soma/N;
return(aux);
}

int main()
{
int i,venda1[N],venda2[N],total[N];
float m1,m2;
printf(" digite quantidades vendidas no primeiro dia\n");
leitura(venda1);
printf(" digite quantidades vendidas no segundo dia\n");
leitura(venda2);
soma_vetor(venda1,venda2,total);
printf(" As quantidades vendidas nos 2 dias\n" );
escreve(total);
m1=media(venda1);
printf(" A media de vendas do primeiro dia: %0.1f\n",m1 );
m2=media(venda2);
printf(" A media de vendas do segundo dia: %0.1f\n",m2 );
}
```

Código-fonte 30 – Segundo programa com passagem de parâmetros do tipo vetor
Fonte: Elaborado pelo autor (2020)

CONCLUSÃO

A avalanche de informações no presente capítulo permitiu conhecer uma linguagem de programação, ao mesmo tempo simples e poderosa, abordando elementos que compõem a sua sintaxe básica, e finalizando com instruções mais sofisticadas como struct e ponteiros, característicos da linguagem.

A partir de agora, é preciso praticar para que a maturidade computacional exigida na resolução de problemas seja conquistada. Isso somente irá acontecer se houver o treino adequado para elaborar programas mais e mais complexos.

O próximo passo é com você!

REFERÊNCIAS

CELES, Waldemar. **A importância e as vantagens de saber programar em linguagem C**. 2016. Disponível em: <[A importância e as vantagens de saber programar em linguagem C – FAETERJ-Rio](#)>. Acesso em: 27 abr. 2020.

DEITEL, P.; DEITEL, H. **C: como programar**. São Paulo: Pearson, 2011.

HOWARD, M.; LEBLANC, D. **WRITING Secure Code**. 2. ed. USA: Microsoft Press, 2002.

MIZAH, V. V. **Treinamento em Linguagem C**. São Paulo: Pearson, 2008.

TANENBAUM, A. S. **Organização estruturada de computadores**. São Paulo: Pearson Prentice Hall, 2013.

TIOBE. Índice TIOBE. Disponível em: <<https://www.tiobe.com/tiobe-index/>>. Acesso em: 10 abr. 2020.

ZIVIANI, N. **Projeto de algoritmos com implementações em Pascal e C**. São Paulo: Pioneira, 2000.